

Using The Observer Pattern To Assess Parking Changes

ICT 4315: Object Oriented Methods and Programming II

for

Master of Science

Information Technology

Kalika Browder

University of Denver College of Professional Studies

May 3, 2026

Faculty: Nathan Braun, MBA

Director: Cathie Wilson, M.S.

Dean: Michael J. McGuire, MLS

This assignment extends previous iterations of my University Parking System by implementing the Observer (Publish-Subscribe) design pattern to handle vehicle entry and exit events. The core problem that is being solved with this new iteration is decoupling charge assessment from the parking lot itself. Previously, ParkingLot was responsible for directly knowing about ParkingOffice and calling charge methods on it... tight coupling that makes the system harder to maintain and extend. The Observer pattern breaks that dependency by having ParkingLot simply just publish an event when something happens, and then letting independent observer objects react however they need to.

This implementation introduces four new elements: ParkingAction (the observer interface), ParkingEvent (the event payload), ParkingObserver (the concrete observer that records transactions), and modifications to ParkingLot to make it a proper Subject.

Design

The key relationships in this new implementation are:

ParkingLot which is the Subject (Observable). It maintains a List<ParkingAction> called observers and provides three Subject methods including addObserver(ParkingAction), removeObserver(ParkingAction), and notifyObservers(ParkingEvent). When enter(ParkingPermit) or exit(ParkingPermit) is called, the lot creates a ParkingEvent capturing the lot, the permit, the timestamp, and the event type (ENTER or EXIT). It then calls

notifyObservers() to broadcast it. Critically, ParkingLot does not import or reference ParkingObserver at all, as it only knows the ParkingAction interface.

ParkingAction is the Observer interface. It defines a single method: void update(ParkingEvent event). Any class implementing this interface can subscribe to any ParkingLot. The interface is intentionally minimal so that radically different kinds of observers (a fullness tracker, a where's-my-car tracker, a billing system) can all coexist in the ParkingLot file without needing to know about any of them.

ParkingEvent is the event payload. It is an immutable data object carrying the four pieces of information that every observer needs: the ParkingLot that fired the event, the ParkingPermit involved, the Instant timestamp, and an EventType enum (ENTER or EXIT). Passing a structured event object, rather than separate parameters, makes the observer callback futureproof, where new fields can be added to ParkingEvent without changing method signatures.

ParkingObserver is the concrete Observer. Its constructor accepts a TransactionManager and a ParkingOffice, iterates over all lots in the office and calls lot.addObserver(this) on each one. When update(ParkingEvent event) fires, the observer inspects both the lot's ScanType and the event's EventType to decide whether to charge: ENTRY_ONLY lots charge on ENTER, ENTRY_EXIT lots charge on EXIT. If the condition matches, it calls transactionManager.park(timestamp, permit, lot) to record the transaction.

TransactionManager remains an independent actor in this context. It knows nothing about the Observer pattern and receives a park() call with the three required arguments and

records a ParkingTransaction. This separation of concerns is intentional: the observer decides when to charge, while the transaction manager decides how to record it.

Pattern Analysis

The implementation matches the formal definition of the Observer pattern closely. The Gang of Four definition states the following: “Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.” Each element maps directly to the following:

- The Subject is ParkingLot. Its state change is a vehicle scan (entry or exit).
- The Observer interface is ParkingAction with its update(ParkingEvent) method.
- The ConcreteObserver is ParkingObserver, which reacts to the notification by recording a transaction.
- The one-to-many relationship is established by ParkingLot’s observers list... one lot can have arbitrarily many observers.
- The decoupling is enforced by the interface: ParkingLot only ever references ParkingAction, never ParkingObserver directly.

I noted that the assignment guideposts stated something along the lines of the observer pattern’s beauty lies in it’s ability to observe the Subject and perform any actions in response, with the Subject being aware, in any class. This is what is achieved in this new iteration of my

Parking system. A second observer (like a fullness counter, for example), could be registered with `addObserver()` without modifying a single line of `ParkingLot`.

One subtle distinction is that rather than passing raw primitives to `update()`, a `ParkingEvent` object is passed. This is a common real-world refinement that keeps the notification contract stable as the system grows, and it is consistent with Java's event model used in Swing and other frameworks.

Challenges and Solutions

The most meaningful design challenge was deciding *when* to charge. For example was it on entry or on exit? Then it was making that logic live in the right place. An initial instinct might be to put the `ENTRY_ONLY` vs. `ENTRY_EXIT` check inside `ParkingLot` before firing the event, so that the lot only notifies observers "when a charge is warranted." This would work, but it leaks business logic (billing rules) back into the lot, defeating the decoupling purpose of the system implementation.

The better solution was to place the check inside `ParkingObserver.update()`. It keeps the lot's job simple: fire an event whenever *any* scan happens. The observer decides what to do with it. This means `ENTER` events on `ENTRY_EXIT` lots flow through to observers and are simply ignored by `ParkingObserver`. If a future observer wanted to react to every entry (to count occupancy, for instance), it could do so with no changes to `ParkingLot`!

A second consideration for me when designing this was thread safety in `notifyObservers()`. The implementation copies the observer list into a temporary `ArrayList` before iterating (`new ArrayList<>(observers)`). This prevents a `ConcurrentModificationException` in cases where an observer removes itself during notification.

Finally, null safety was addressed throughout, and was probably my toughest consideration in the matter. `addObserver()` silently ignores null and duplicate registrations. `enter()` and `exit()` return early on null permits. `update()` returns immediately on a null event. These guards prevented cascading failures and made the null-safety unit tests straightforward to write and pass, in my opinion.

Overall, this implementation was a challenge for me, more so than in previous weeks. I felt it took a lot of consideration to decide how to implement the Observer pattern correctly, but I do think I got there eventually and can say that I'm proud of what I've accomplished.